

OBJETIVOS

- Primeiros passos para o desenvolvimento de um SO e testes em um ambiente de máquina virtual.
- Geração de *kernels* executáveis em formatos “.bin” e como ISO.
- Compreensão básica da estrutura de *boot*.
- Operações de I/O em tela em baixo nível.

TAREFA 1: PREPARAÇÃO DO SISTEMA E CRIAÇÃO DO SEU PRIMEIRO KERNEL (de 16/01/2025 a 19/01/2025)

O objetivo desta prática será, em última análise, fornecer a percepção das complexidades envolvidas no desenvolvimento de um Sistema Operacional (SO). Muito embora os resultados descritos possam ser obtidos em outros SOs, recomenda-se o uso do Linux devido a disponibilidade e licenciamento deste sistema. Além disso, será utilizada a linguagem C e um pouco de Assembly. Dessa forma, os próximos passos representam a preparação do SO Linux para essa prática de desenvolvimento de SO.

O primeiro passo será a criação de um *shell script* para o ajuste/instalação de alguns pacotes necessários para esta prática de laboratório. Crie este arquivo com um editor de texto e o chame de `setup.sh`. A maneira como este arquivo deve ser editado é exibida no Código 1.

```
#!/bin/sh
sudo apt-get update
sudo apt-get install xorriso
sudo apt-get install grub-common
sudo apt-get install grub2-common
sudo apt-get install grub-pc
sudo apt-get install gcc
sudo apt-get install nasm
sudo apt-get install virtualbox
sudo apt-get install qemu
sudo apt-get install gedit #opcional
```

Código 1: conteúdo do arquivo `setup.sh`.

Após a criação de `setup.sh`, verifique suas propriedades de leitura/escrita/execução e ajuste-as de maneira apropriada de modo que o *shell script* possa ser executado (**dica:** você pode fazer isso via interface gráfica – *graphical user interface* ou GUI, ou via linha de comando – *command line interface* ou CLI). Se optar por fazer este ajuste via CLI, pesquise o comando `chmod`. Em seguida,

execute o *shell script* e aguarde o seu término. Talvez alguns pacotes já estejam instalados.

No Código 1 temos:

- `xorriso`, será responsável pela criação da imagem (.iso) de nosso SO;
- `grub` é o *boot loader* do sistema. A suíte de ferramentas instalada (`grub` ou `grub2`), também será responsável por auxiliar na criação das entradas necessárias para a atuação do *boot loader* (ex. configuração do arquivo `grub.cfg`);
- `gcc` auxiliará na compilação do código C;
- `nasm` auxiliará na compilação do código Assembly, necessário para a edição dos pontos de entrada do *kernel*
- `virtualbox`, nos permitirá testar as versões do SO que serão criadas (em formato ISO);
- `qemu` permitirá o teste rápido do kernel do SO (em formato .bin), sem a necessidade de gerar uma imagem ISO a cada passo; e
- `gedit` é um editor de texto simples (você pode usar o editor que preferir).

A seguir, crie uma pasta chamada “so” e iremos prepará-la para o projeto. Nesta pasta, crie um arquivo chamado `kernel.asm`. Este será o ponto de entrada (*entry point*) do *kernel* do SO que está sendo criado. Crie também os arquivos chamado `kernel.c` e `link.ld`. Crie outro *shell script* chamado `build.sh` e faça os ajustes apropriados para torná-lo um arquivo executável. Estes arquivos serão explicados posteriormente.

Neste momento, vale observar que, não iremos construir o *bootloader*. O *bootloader* é um código muito pequeno que fica alojado no primeiro setor do disco que irá carregar o SO (*boot drive*). Esse trecho de código aponta para um endereço específico para continuar executando. Esse endereço é específico para o `kernel.asm`, o qual é o ponto de entrada de nosso *kernel*. O *kernel*, como já foi explicado em outras ocasiões, é a personificação do SO.

O *kernel* é dividido em duas partes: a primeira parte é o ponto de entrada, o qual deve estar localizado em um endereço específico na memória. Esse ponto de entrada irá apontar para a execução do código do *kernel*. De forma simplificada teremos algo como: *bootloader* → `kernel.asm` → `kernel.c`. Perceba que os arquivos mencionados são arquivos de texto. Teremos que inserir código nestes e então, compilá-los com as ferramentas apropriadas.

Para começar, abra o arquivo `kernel.asm` e edite-o conforme destacado no Código 2.

O Código 2 representa o ponto de entrada do *kernel*. Neste momento, você não precisa compreender totalmente este código, o qual será tratado pelo *linker*. Entretanto, alguns pontos mais importantes serão destacados:

- `bits 32` não é uma instrução em Assembly. É uma diretiva para o NASM *assembler* que especifica que este deve gerar código para ser executado em um processador em modo de 32

bits. Não é obrigatório no exemplo, mas é incluído como parte da boa prática de ser explícito;

```
bits 32
section      .text
            align 4
            dd 0x1BADB002
            dd 0x00
            dd - (0x1BADB002+0x00)

global      start
extern      kmain
start:
    cli
    call kmain
    hlt
```

Código 2: conteúdo do arquivo *kernel.asm*.

- a segunda linha (`section .text`) inicia a seção de texto (a.k.a seção de código). Aqui é onde se coloca todo o código. Estes são definidos usando-se a diretiva `dd`;
- a partir da terceira linha, são inseridas especificações para que se possa utilizar o GRUB (GRand Unified Bootloader), por isso o *kernel* é criado com suporte a *multiboot*. De acordo com a especificação, o *kernel* deve conter um cabeçalho nos primeiros 8 KB. Além disso, o cabeçalho deverá possuir três campos que estão alinhados em 4 bytes (`align 4`);
- `0x1BADB002` é o chamado “número mágico” (*magic number*). Este é o número que o *bootloader* deve encontrar para se assegurar que este é um SO que aceita *multi boot*.
- o próximo campo é destinado a *flags* e será ignorado (`dd 0x00`);
- o último campo é o *checksum*. Este, ao ser somado ao campo de *magic number* e *flags*, deve dar 0.
- será criada uma função chamada `start`. Esta função é definida como `global` para que possa ser acessada usando-se o *linker*. A execução desta função será iniciada com o *linker*.
- por último é criada uma nova função externa chamada `kmain`, a qual estará em `kernel.c`.
- na função `start`, a primeira linha `cli`, limpa as interrupções. Neste contexto, as interrupções são alguns serviços oferecidos pela BIOS, os quais realizam algumas operações básicas, como mapeamentos para a tela, rastreamento de teclado etc. Ao passar para o modo de 32 bits, ou modo protegido, não é mais possível usar estes serviços. Assim, é necessário limpar as interrupções, já que estas não serão usadas novamente, e assim, liberar espaço para o código C que será carregado em seguida.
- a linha `call kmain` manda o processador continuar sua execução na função `kmain`, a qual será definida em `kernel.c`.

- a última linha `hlt`, “pausa” a execução da CPU considerando o endereço corrente.

Seria ótimo escrever tudo em C, entretanto, não há como evitar um pouco de Assembly. Este pequeno trecho de código Assembly serve para invocar uma função externa, a qual será escrita em C, e então interromper o fluxo do programa. Para que isso funcione conforme o esperado, será usado posteriormente um *script* de ligação (*linker script*), o qual liga os arquivos objeto para produzir o executável final do *kernel* (maiores detalhes posteriormente). Neste *script* de ligação, será especificado explicitamente que queremos que nosso arquivo binário seja carregado no endereço `0x100000` (conforme mencionado na seção de PRELÚDIO). Este endereço, conforme explicado anteriormente, é onde se espera que o *kernel* será alojado. Assim, o *bootloader* irá cuidar de disparar o ponto de entrada do *kernel*.

Neste ponto, falta apenas a implementação da função `kmain`, a qual será inserida dentro do arquivo `kernel.c` (Código 3). Esta função irá apenas imprimir um caractere na tela do computador (‘X’ na cor verde) como forma de demonstrar o funcionamento de nosso código. Além disso, neste momento, trataremos de manter o foco nos procedimentos de compilação/geração dos códigos/arquivos necessários para um *boot* real para o nosso futuro SO (ultra simples). Neste momento, pode-se dizer que temos um SO funcional.

```
kmain()
{
    char *vidmem = (char*) 0xb8000;

    vidmem[0] = 'X';
    vidmem[1] = 0x02; /* cor do caractere: verde */
}
```

Código 3: conteúdo do arquivo *kernel.c*.

Para compilar este código, é necessário se compilar inicialmente o código `kernel.asm`, o qual representa o ponto de entrada de nosso kernel. Na sequência, também precisamos compilar o arquivo `kernel.c`, o qual será indicado para execução a partir do código `kernel.asm`. Após se compilar estes dois arquivos, iremos combiná-los, com o intuito de se construir um binário final para execução. Para compilar estes códigos, recomenda-se o uso do script apresentado no Código 4.

```
#!/bin/sh
nasm -f elf32 kernel.asm -o kasm.o
gcc -m32 -c -w kernel.c -o kc.o
ld -m elf_i386 -T link.ld -o kernel.bin kasm.o kc.o
```

Código 4: conteúdo do *shell script* `build.sh`.

Perceba no Código 4 que o arquivo `kernel.asm` é compilado utilizando-se o compilador Netwide Assembler (`nasm`), um *assembler* 80x86 portátil. Este código é gerado no formato

Executable and Linking Format ou Extensible Linking Format (ELF) de 32 bits. Este padrão é flexível, extensível e multiplataforma, não ligado a qualquer unidade central de processamento (UCP ou CPU, derivado do termo em inglês) específica ou arquitetura de conjunto de instruções. Isto o possibilitou ser adotado por muitos sistemas operacionais diferentes em muitas plataformas de hardware diferentes. Em seguida, o arquivo `kernel.c` é compilado com o `gcc`, utilizando a opção `-m32` para geração de código de 32 bits. Após a geração dos dois arquivos binários resultantes (`kasm.o` e `kc.o`), falta realizar a união destes, e claro, esta deve manter a ordem com o *entry point* sendo inserido primeiro (i.e. `kasm.o`).

Para realizar a união dos arquivos binários gerados, usaremos o *linker*. Assim sendo, é chegada a hora de especificar o arquivo `link.ld` (Código 5). O formato de saída após a junção de ambos os arquivos binários é ELF de 32 bits e a arquitetura é explicitamente especificada como i386. `ENTRY (start)` representa o *entry point* especificado em `kernel.asm` (lembra-se de `global start`?). Assim, o *kernel* irá iniciar a execução desta função e no ponto `call kmain` chama-se a função em `kernel.c`. após isso, o *kernel* irá pausar sua execução (*halt* em `hlt` em `kernel.asm`). As seções (SECTIONS) foram definidas anteriormente. A seção “.” representa a posição (i.e. endereço) a partir da qual o primeiro bit de código será armazenado. `.text`, aparece em `kernel.asm`. As demais seções serão explicadas à medida que forem necessárias (no momento serão ignoradas).

```
OUTPUT_FORMAT(elf32-i386)
ENTRY(start)
SECTIONS
{
    . = 0x100000;
    .text : { *(.text) }
    .data : { *(.data) }
    .bss  : { *(.bss)  }
}
```

Código 5: conteúdo do arquivo `link.ld`.

A combinação dos arquivos binários é então realizada a partir da execução do *linker* (comando `ld`) na quarta linha do Código 4. Para tanto, o arquivo `link.ld` é necessário e a ordem na qual serão combinados os arquivos binários deve ser explicitada na linha de comando. A saída será o arquivo `kernel.bin`.

Para testar o *kernel* gerado, sugere-se o uso do software emulador `qemu`. O comando a ser executado é: `qemu-system-i386 --kernel kernel.bin` (em caso de variação deste, pesquise). Caso deseje, você pode inserir esta linha de código para execução automática do *kernel* (após sua geração) como parte do Código 4 ou ainda, você pode criar um *script* de execução separado. Para uma questão de completude, o próximo passo é a conversão do arquivo

`kernel.bin` em um arquivo ISO, de modo a permitir a execução de nosso SO simples, por exemplo, no VirtualBox.

Para a geração do arquivo ISO, será necessário criar uma estrutura de diretórios, da seguinte forma: crie um diretório (ex. *simple*). Internamente ao diretório criado, crie um novo diretório chamado *boot* (ex. *simple/boot*) e por último, crie um diretório chamado *grub* (ex. *simple/boot/grub*). Dentro do diretório *grub*, crie um arquivo vazio chamado *grub.cfg*. Abra esse arquivo e insira o Código 6. Para que esse arquivo de configuração funcione corretamente, você deverá inserir o arquivo `kernel.bin` gerado anteriormente, no diretório *boot* criado.

```
set default = 0
set timeout = 0

menuentry "simpleOS" {
    set root='(hd96) '
    multiboot /boot/kernel.bin
}
```

Código 6: conteúdo do arquivo *grub.cfg*.

No arquivo *grub.cfg*, ajusta-se a execução deste SO como padrão (`set default = 0`), e o tempo aguardado para sua inicialização é ajustado em 0 segs (`set timeout = 0`). Além disso, é criada uma entrada de menu, para a escolha do SO, chamado de “*simpleOS*”. Nessa entrada, define-se o dispositivo de *boot* como sendo o CD-ROM (`set root='(hd96) '`), carregado a partir do arquivo `kernel.bin`, o qual está localizado na pasta *boot*.

Caso todos os passos tenham sido executados adequadamente, você agora fará a geração do ISO mediante o seguinte comando: “`grub-mkrescue -o simple.iso simple`”. Para testar a sua imagem ISO, crie uma máquina virtual no VirtualBox, com 64 MB de RAM, com descrições de “Other” para o tipo do SO e “Other/Unknown” para a versão (32 bits desconhecido). Para finalizar, configure esta máquina virtual para ser inicializada com um CD de boot, apontando para a imagem ISO criada (*simple.iso*). Para simplificar atividades futuras, você também pode inserir o código para geração do ISO como parte do Código 4.

Vale observar que, durante as atividades a serem realizadas, pode ser que não necessitemos realizar todos os passos apresentados no Código 4. Assim, é possível usar o símbolo de comentário (`#`) para evitar que determinadas partes do *script* sejam executadas. Outra dica, é mudar o local de saída do `kernel.bin`, de modo que este arquivo seja gerado diretamente dentro do diretório *boot* para a geração apropriada do arquivo ISO. Todas essas modificações são opcionais e ficam a critério do aluno. **Entregue no SIGAA todos os códigos e scripts resultantes desta tarefa para análise.**